16-bit ALU Design in VHDL

Last time, I introduced the N-bit adder design in Verilog, which is a part of a 16-bit ALU design I will present today. The 16-bit ALU is a core combinational component of the processing unit in the coprocessor I introduced in the previous post.

Full VHDL code for 16-bit ALU together with testbench will be presented in this VHDL project.

The instruction set of the 16-bit ALU is as follows:
1. ADD: ABUS + BBUS -> ALUOUT
2. SUB: ABUS - BBUS -> ALUOUT
3. AND: ABUS & BBUS -> ALUOUT
4. OR: ABUS | BBUS -> ALUOUT
5. XOR: ABUS ^ BBUS -> ALUOUT
6. NOT: ~ABUS -> ALUOUT
7. MOV: ABUS -> ALUOUT
The addition/ subtraction of the 16-bit ALU is designed and implemented using the Verilog N-bit Adder.
VHDL code for 16-bit ALU:

```
-- fpga4student.com: FPGA projects, Verilog projects, VHDL projects
-- VHDL project: VHDL code for 16-bit ALU
-- Top level VHDL code for 16-bit ALU
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
-- 16-bit ALU
entity ALU is
 port (
   ABUS: in std_logic_vector(15 downto 0); -- ABUS data input of the 16-bit ALU
   BBUS: in std_logic_vector(15 downto 0); -- BBUS data input of the 16-bit ALU
   ALUctrl: in std_logic_vector(3 downto 0); -- ALUctrl control input of the 16-bi
   ALUOUT: out std_logic_vector(15 downto 0)  -- 16-bit data output of the 16-bit
   );
end ALU;

architecture Behavioral of ALU is
-- N-bit Adder in Verilog
component N_bit_adder is
generic (
    N: integer:=32
   );
port( input1: in std_logic_vector(N-1 downto 0);
  input2: in std_logic_vector(N-1 downto 0);
  answer: out std_logic_vector(N-1 downto 0)
 );
end component N_bit_adder;
signal BBUS_not: std_logic_vector(16-1 downto 0);
signal tmp_out1: std_logic_vector(16-1 downto 0);
signal tmp_out2: std_logic_vector(16-1 downto 0);
```

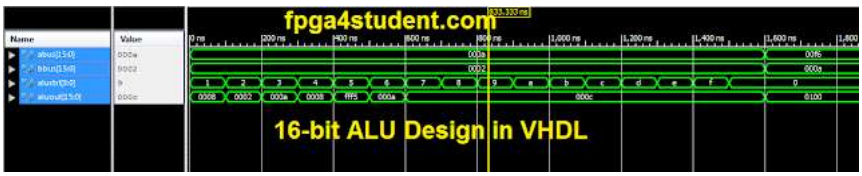
```
signal tmp: std_logic_vector(16-1 downto 0);
begin
-- instantiate Verilog N-bit Adder in VHDL code
u1_N_bit_adder: N_bit_adder generic map ( N => 16) -- ABUS + BBUS
     port map( input1 => ABUS, input2 => BBUS,answer => tmp_out1 );
u2_N_bit_adder: N_bit_adder generic map ( N => 16) -- ABUS + (~BBUS)
     port map( input1 => ABUS, input2 => BBUS_not,answer => tmp_out2 );
u3_N_bit_adder: N_bit_adder generic map ( N => 16) -- ABUS + (~BBUS) + 1 = ABUS -
     port map( input1 => tmp_out2, input2 => x"0001",answer => tmp );
BBUS_not <= not BBUS;
-- Other instructions of the 16-bit ALU in VHDL
process(ALUctrl,ABUS,BBUS,tmp_out1,tmp)
begin
case(ALUctrl) is
 when "0000" =>  ALUOUT <= tmp_out1;  -- ADD
 when "0001" =>  ALUOUT <= tmp ;-- SUB
 when "0010" =>  ALUOUT <= ABUS and BBUS; -- AND
 when "0011" =>  ALUOUT <= ABUS or BBUS; -- OR
 when "0100" =>  ALUOUT <= ABUS xor BBUS; -- XOR
 when "0101" =>  ALUOUT <= not ABUS; -- NOT
 when "0110" =>  ALUOUT <= ABUS; -- MOVE
 when others => ALUOUT <= tmp_out1;
 end case;
end process;

end Behavioral;
```

Testbench VHDL code for 16-bit ALU:

```
-- fpga4student.com: FPGA projects, Verilog projects, VHDL projects
-- VHDL project: VHDL code for 16-bit ALU
-- Testbench VHDL code for 16-bit ALU
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
use IEEE.std_logic_unsigned.all;
-- Testbench
ENTITY tb_ALU IS
END tb_ALU;

ARCHITECTURE behavior OF tb_ALU IS

    -- Component Declaration for the 16-bit ALU

    COMPONENT ALU
    PORT(
         ABUS : IN  std_logic_vector(15 downto 0);
         BBUS : IN  std_logic_vector(15 downto 0);
         ALUctrl : IN  std_logic_vector(3 downto 0);
         ALUOUT : OUT  std_logic_vector(15 downto 0)
        );
    END COMPONENT;

   --Inputs
   signal ABUS : std_logic_vector(15 downto 0) := (others => '0');
   signal BBUS : std_logic_vector(15 downto 0) := (others => '0');
   signal ALUctrl : std_logic_vector(3 downto 0) := (others => '0');

  --Outputs
   signal ALUOUT : std_logic_vector(15 downto 0);

BEGIN

 -- Instantiate the 16-bit ALU
```

```
   uut: ALU PORT MAP (
          ABUS => ABUS,
          BBUS => BBUS,
          ALUctrl => ALUctrl,
          ALUOUT => ALUOUT
        );
   stim_proc: process
   begin
      ABUS <= x"000A";
  BBUS <= x"0002";
  ALUctrl <= x"0";
  -- change ALU Control input
   for i in 0 to 15 loop
  ALUctrl <= ALUctrl + x"1";
  wait for 100 ns;
   end loop;
      ABUS <= x"00F6";
  BBUS <= x"000A";
      wait;
   end process;

END;
```

Simulation Waveform for 16-bit ALU:



The simulation waveform shows the correct operations of the 16-bit ALU. After fully verifying the VHDL 16-bit ALU, the 16-bit ALU will be used in the processing unit of the co-processor in the next post.

Recommended VHDL projects:

1. What is an FPGA? How VHDL works on FPGA
2. VHDL code for FIFO memory
3. VHDL code for FIR Filter
4. VHDL code for 8-bit Microcontroller
5. VHDL code for Matrix Multiplication
6. VHDL code for Switch Tail Ring Counter
7. VHDL code for digital alarm clock on FPGA
8. VHDL code for 8-bit Comparator
9. How to load a text file into FPGA using VHDL
10. VHDL code for D Flip Flop
11. VHDL code for Full Adder
12. PWM Generator in VHDL with Variable Duty Cycle
13. VHDL code for ALU
14. VHDL code for counters with testbench
15. VHDL code for 16-bit ALU
16. Shifter Design in VHDL
17. Non-linear Lookup Table Implementation in VHDL
18. Cryptographic Coprocessor Design in VHDL
19. Verilog vs VHDL: Explain by Examples
20. VHDL Code for Clock Divider on FPGA
21. How to generate a clock enable signal instead of creating another clock domain
22. VHDL code for debouncing buttons on FPGA
23. VHDL code for Traffic light controller
24. VHDL code for a simple 2-bit comparator
25. VHDL code for a single-port RAM
22. VHDL code for debouncing buttons on FPGA
23. VHDL code for Traffic light controller

24. VHDL code for a simple 2-bit comparator
25. VHDL code for a single-port RAM
26. VHDL code for Car Parking System using FSM
27. VHDL coding vs Software Programming
28. VHDL code for MIPS Processor
29. VHDL code for Moore FSM Sequence Detector
30. VHDL code for Seven-Segment Display on Basys 3 FPGA

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here
Copyright © 2016-2020 FPGA4student.com All Rights Reserved.